

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	Vesting, Governance	Documentation quality	Medium	
Timeline	2025-07-21 through 2025-07-24	Test quality	Medium	
Language	Solidity	Total Findings	9	9 Fixed: 8 Acknowledged: 1
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review	High severity findings ⓘ	0	
Specification	None	Medium severity findings ⓘ	4	4 Fixed: 4
Source Code	https://github.com/hemilabs/veHEMI ⓘ #be7a578 ⓘ	Low severity findings ⓘ	4	4 Fixed: 3 Acknowledged: 1
Auditors	- Julio Aguilar Auditing Engineer - Rabib Islam Senior Auditing Engineer - Paul Clemson Auditing Engineer	Undetermined severity findings ⓘ	0	
		Informational findings ⓘ	1	1 Fixed: 1

Summary of Findings

The codebase implements a voting escrow system modeled after Curve's `veCRV` , where users lock `HEMI` tokens for up to four years in exchange for voting power that decays linearly over time. The `VeHemi` contract manages token locks as NFTs, and the `VeHemiVoteDelegation` contract enables users to delegate their voting power to other token holders without transferring ownership. The system supports creating locks, increasing locked amounts or durations, transferring tokens (when permitted), and withdrawing tokens after lock expiry. The contracts are well-structured, and the code quality is high. The test suite is thorough, covering lock creation, delegation, voting power calculation, and various edge cases.

The audit uncovered several issues in the `VeHemiVoteDelegation` contract, including the use of potentially stale values when calculating voting weights, the risk of voting power becoming inaccessible if a delegator's lock expires, and the automatic transfer of delegated votes when an NFT is transferred. Additionally, if a delegatee's NFT is forfeited, any voting power delegated to it is effectively wasted. We recommend addressing these, along with other low- and informational-severity issues, before deployment.

Fix-Review Update: All issues have been addressed by implementing fixes or acknowledging them. The fixes for `VEH-2`, `VEH-3`, and `VEH-4` involved modifying functionality to assign delegation to an address rather than to a token ID. The team also proactively discovered and resolved an underflow bug that could have significantly increased the voting power of a previous delegatee when switching delegates. Additionally, improvements to the test suite have further strengthened the robustness of the codebase.

ID	DESCRIPTION	SEVERITY	STATUS
VEH-1	Potential Stale Values Used to Calculate Voting Weight in the <code>VeHemiVoteDelegation</code> Contract	• Medium ⓘ	Fixed
VEH-2	Potential for Dead Voting Weight if the Delegator's Lock Expires	• Medium ⓘ	Fixed
VEH-3	Transferring NFT Also Transfers Delegated Votes	• Medium ⓘ	Fixed
VEH-4	Delegatee Forfeiture Wastes Delegator's Votes	• Medium ⓘ	Fixed

ID	DESCRIPTION	SEVERITY	STATUS
VEH-5	<code>VeHemi</code> Contract Should Use <code>ReentrancyGuardUpgradeable</code>	• Low ⓘ	Fixed
VEH-6	Missing Signature Malleability Check when Using <code>ecrecover</code> Precompile	• Low ⓘ	Fixed
VEH-7	Global Point History Can Contain Temporarily Stale Amount	• Low ⓘ	Fixed
VEH-8	Unsafe Storage Management	• Low ⓘ	Acknowledged
VEH-9	<code>totalSupply()</code> Logic Deviates From Erc-721	• Informational ⓘ	Fixed

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

i

Disclaimer

Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
 1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
 2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
 3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:
 1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
 2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

Files Included

Repo: [https://github.com/hemilabs/veHEMI\(39bea4a018205de42b1ffe365fd9022075b239fd\)](https://github.com/hemilabs/veHEMI(39bea4a018205de42b1ffe365fd9022075b239fd)) Files: `src/*.sol`
Repo: <https://github.com/hemilabs/veHEMI>

Files Excluded

Repo: [https://github.com/hemilabs/veHEMI\(39bea4a018205de42b1ffe365fd9022075b239fd\)](https://github.com/hemilabs/veHEMI(39bea4a018205de42b1ffe365fd9022075b239fd)) Files: `./test/*` `./lib/*`

Operational Considerations

- The contracts follow an upgradeable pattern, which makes it possible to patch vulnerabilities but also introduces risk if a flawed or malicious implementation is upgraded. It is important that the upgrade process is secured by a robust test suite and strong multi-signature governance.
- The voting system itself is out of scope of this audit. It will be important to ensure that the voting power of a token ID at the given timestamp of the vote is correctly calculated. The voting system should also implement additional verification checks to ensure legitimacy.
- The forfeit admin has significant authority to seize locked tokens. This power can also be used to manipulate voting power as discussed in [VEH-4](#). Therefore, a compromise of the admin key could result in abuse of this power. We recommend using a multi-sig.
- The `RewardDistributor` is out of scope of this audit.
- If the `voteDelegation` contract is changed in `VeHemi`, all existing vote delegation information will be lost.
- Try-catch blocks that fail silently can run into out-of-gas issues because of the 63/64ths rule. It is important that there is sufficient function logic preceding any such try/catch blocks to ensure the entire transaction runs out of gas and reverts in such scenarios.

Key Actors And Their Capabilities

VeHemi

1. Contract Owner
 - Can update the reward distributor contract
 - Can update the vote delegation contract
 - Can update the forfeit admin address
 - Controls the upgrade process
2. Forfeit Admin
 - Can seize tokens from locks marked as forfeitable
3. Token Holders
 - Can create locks for themselves or others
 - Can increase lock amounts
 - Can increase lock durations
 - Can withdraw tokens after lock expiry
 - Can transfer tokens if the token is transferable
4. Reward Distributor
 - External contract that can distribute rewards to token holders
5. Vote Delegation Contract
 - Manages delegation of voting power between tokens

VeHemiVoteDelegation

1. Token Holders
 - Can delegate their voting power to other token holders
 - Can delegate via signature using `delegateBySig()`
 - Can query their own and others' voting power
2. VeHemi Contract
 - Can call `delegate()` on behalf of any token
 - Used during transfers to reset delegations
3. Any user
 - Can call `getVotes()` and `getPastVotes()` to check voting power
 - Can call `writeNewCheckpointForExpiredDelegations()` to update checkpoints

Findings

VEH-1

Potential Stale Values Used to Calculate Voting Weight in the `VeHemiVoteDelegation` Contract

● Medium ⓘ Fixed



Update

Marked as "Fixed" by the client.
Addressed in: `e43e96ac4e8d6d032ee1a2b6b0cfc556446e7ea4` .

File(s) affected: `VeHemiVoteDelegation.sol`

Description: When a user delegates their voting power in `VeHemiVoteDelegation` their token lock information is stored in the `VeHemiVoteDelegation` contract. However, in the scenario where the user alters the state of their original lock (such as by increasing the locked amount or lock duration) in the `VeHemi` contract, the state of their lock is not updated within the `VeHemiVoteDelegation` contract. This means that unless the user manually redelegates, the weight calculations made in `VeHemiVoteDelegation` will use stale lock information to calculate the delegatee's total voting power.

Recommendation: When a `tokenId` has its lock amount or duration increased, consider redelegating to their existing delegatee to ensure the state is correctly updated in the `VeHemiVoteDelegation` contract.

VEH-2

Potential for Dead Voting Weight if the Delegator's Lock Expires

• Medium ⓘ

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `090aef60ea416ca612e4913cc1af12647094961a` .

The client provided the following explanation:

The project team decided to change this functionality by assigning delegation to the wallet instead of the `tokenId`. Issues [VEH-2](#), [VEH-3](#), and [VEH-4](#) are addressed with this update. The logic and test cases have been updated accordingly.

File(s) affected: `VeHemi.sol`, `VeHemiVoteDelegation.sol`

Description: When calculating voting power in the `_getPastVotes()` function, there is a check to ensure that the lock of a given `tokenId_` has not expired. In this scenario, the total voting power of `tokenId_` will always be zero, even if other users have delegated to this token ID. This creates a potential scenario where users' voting power is temporarily stuck inside an expired lock. This means that users would not be able to redelegate their voting power until the next delegation checkpoint, meaning a vote would take place despite a large amount of dead voting power.

Recommendation: Consider either allowing expired locks to vote on behalf of their delegates, or returning voting power to the delegates if their delegatee's lock has expired.

VEH-3 Transferring NFT Also Transfers Delegated Votes

• Medium ⓘ

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `090aef60ea416ca612e4913cc1af12647094961a` .

The client provided the following explanation:

The project team decided to change this functionality by assigning delegation to the wallet instead of the `tokenId`. Issues [VEH-2](#), [VEH-3](#), and [VEH-4](#) are addressed with this update. The logic and test cases have been updated accordingly.

File(s) affected: `VeHemiVoteDelegation.sol`

Description: Suppose Alice delegates her NFT's votes to Bob's NFT. Bob's NFT thereby acquires the votes of Alice's NFT.

Now, suppose that Bob transfers his NFT to Charlie. Delegations are implemented as `tokenId` to `tokenId` relationships. Because of this detail, upon the transfer of Bob's NFT to Charlie, Alice's votes will be delegated to Charlie.

In typical voting systems, delegation implies an investment of trust from a delegator to a delegatee. However, in `VeHemiVoteDelegation`, someone can have their votes delegated to someone without their ever having been made aware, which may lead to an erosion of trust in the protocol.

Recommendation: Allow those who wish to delegate their NFT's votes the option to either:

1. Upon the transfer of the delegatee's NFT, return the delegated votes to the delegator; or
2. Allow the delegated votes to carry over to the new delegatee.

VEH-4 Delegatee Forfeiture Wastes Delegator's Votes

• Medium ⓘ

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `090aef60ea416ca612e4913cc1af12647094961a` .

The client provided the following explanation:

The project team decided to change this functionality by assigning delegation to the wallet instead of the `tokenId`. Issues [VEH-2](#), [VEH-3](#), and [VEH-4](#) are addressed with this update. The logic and test cases have been updated accordingly.

File(s) affected: `VeHemi.sol`, `VeHemiVoteDelegation.sol`

Description: Suppose Alice delegates her NFT's votes to Bob's NFT. Bob's NFT thereby acquires the votes of Alice's NFT.

However, it is possible that NFTs can be burned by the `forfeitAdmin`, an address that can be determined by the `owner` of `VeHemi`, if they were created with the `forfeitable` property.

Supposing Bob's NFT carries the `forfeitable` property, and the `forfeitAdmin` decides to burn Bob's NFT, Bob would then carry `0` votes. **This would cause Alice's delegation to be ineffective and would render her votes nil.** In order for Alice to recover her votes, she would have to either delegate to herself or another NFT, and this would only take effect after the next delegation checkpoint is reached, like the scenario described in [VEH-2](#).

Recommendation: We recommend one of two approaches to handling delegated votes in the case of an delegatee's NFT being forfeited:

1. Upon forfeiture, return voting power to the delegator; or
2. Allow the delegatee to continue voting with the delegators' voting power.

VEH-5

VeHemi

Contract Should Use

ReentrancyGuardUpgradeable

• Low ⓘ

Fixed

✓

Update

Marked as "Fixed" by the client.
Addressed in: `3203c7e68e3c0947604e4abdf93a1116a8e3ef08`.

File(s) affected: `VeHemi.sol`

Description: The `VeHemi` contract inherits the non-upgradeable `ReentrancyGuard` from OpenZeppelin, not the upgradeable variant `ReentrancyGuardUpgradeable`. This is inconsistent with the contract's upgradeable architecture and can lead to storage layout conflicts and upgrade failures.

Recommendation: Consider inheriting `ReentrancyGuardUpgradeable` rather than `ReentrancyGuard`.

VEH-6

Missing Signature Malleability Check when Using

ecrecover

Precompile

• Low ⓘ

Fixed

✓

Update

Marked as "Fixed" by the client.
Addressed in: `3203c7e68e3c0947604e4abdf93a1116a8e3ef08`.

File(s) affected: `VeHemiVoteDelegation.sol`

Description: The `delegateBySig()` function uses the `ecrecover` precompile to retrieve a signature's signer. However, there is no constraint on the `s` value to limit the potential signature malleability vulnerability where two separate `s` values can produce the same signer.

Recommendation: Consider using the OpenZeppelin `ECDSA` library, which has built-in checks to prevent signature malleability issues.

VEH-7

Global Point History Can Contain Temporarily Stale Amount

• Low ⓘ

Fixed

✓

Update

Marked as "Fixed" by the client.
Addressed in: `682a13a1eb3be4059be2c6b53e96bd5734495043`.

File(s) affected: `VeHemi.sol`

Description: In the `_checkpoint()` function, the `_lastPoint.amount` is assigned to be the HEMI balance of the `VeHemi` contract. However, when the `_checkpoint()` function is called in the course of depositing or withdrawing from the contract, the transfer occurs after `_checkpoint()` is called, meaning that `pointHistory[epoch].amount` is potentially stale. Although this does not cause a problem for the `VeHemi` system itself, as this field is not referenced anywhere else, it can cause issues for developers or users who choose to reference this field under the incorrect assumption that it represents the amount of tokens locked in the contract at the end of the block where depositing/withdrawal is done (although it is possible for this to be the case if `checkpoint()` is called after depositing/withdrawing).

Recommendation: Leaving `pointHistory[epoch].amount` equal to zero across all epochs would enhance consistency and clarity of the code, sending a clear signal to any developer or external system that this value is not to be used. Moreover, it would reduce the gas burden on the `_checkpoint()` function, as it is no longer reading the HEMI contract to receive the contract balance.

VEH-8 Unsafe Storage Management

• Low ⓘ Acknowledged

Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

This would exceed the code size limit and require significant refactoring to reduce the bytecode. We decided not to refactor major parts of the core logic. The Storage contract serves as the parent contract. For future upgrades that introduce new storage variables, we will extend the Storage contract. This approach helps avoid storage conflicts.

File(s) affected: `VeHemi.sol`

Description: Since `VeHemi` is an upgradeable contract, its logic can be upgraded by a proxy owner. However, `VeHemi` declares state variables directly and does not use namespaced storage. This leads to an increased level of risk when attempting to upgrade the contract.

For instance, consider the case where V2 of `VeHemi` involves inheriting from a different base contract in addition to `VeHemiStorageV1`. In such a case, it is possible that the additional base contract's storage collides with the storage defined in `VeHemiStorageV1`, causing corruption of variables. This may produce inconsistent post-upgrade behavior, risking loss of funds.

Recommendation: We recommend using [namespaced storage](#) for upgradeable contracts like `VeHemi` in order to reduce the risk of storage collisions during upgrades.

VEH-9 `totalSupply()` Logic Deviates From Erc-721

• Informational ⓘ Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `682a13a1eb3be4059be2c6b53e96bd5734495043`.

File(s) affected: `VeHemi.sol`

Description: [ERC-721](#) defines an "enumeration extension" of the standard that allows publishing the full list of all tokens designated by the contract. Part of it defines `totalSupply()`, a function which counts all the NFTs tracked by the contract.

However, `totalSupply()` is overridden in `VeHemi` to count all the HEMI tokens that have been locked inside the contract. This represents a deviation from the ERC-721 standard, thereby risking unexpected behavior with other programs which rely on standard-compliant behavior, such as other contracts, NFT marketplaces, and block explorers.

Recommendation: Ensure that all functions defined by ERC-721 that are implemented are done so in a way that is compliant with the standard. Use a different function, such as `totalLockedTokens()`, to represent the number of HEMI tokens locked inside `VeHemi`.

Auditor Suggestions

S1 General Improvements

Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `4c662af30e0b12cc9ae49546a56d02343c235a60`.

File(s) affected: `VeHemi.sol`, `VeHemiVoteDelegation.sol`

Description: The following changes could be made to improve the overall quality of the codebase:

1. Consolidate the constant values used in both `VeHemi` and `VeHemiVoteDelegation` into one `Constants.sol` contract.
2. Remove the unnecessary `return` in the `delegateBySig()` function.
3. In `VeHemi.initialize()`, assigning `pointHistory[0].amount = 0` is redundant.
4. The style convention throughout the codebase is to use a trailing underscore for function input parameters; however, this is not adhered to in the `totalSupplyAt()` function.
5. Change `if (forfeitable_) forfeitable[_tokenId] = forfeitable_;` to `if (forfeitable_) forfeitable[_tokenId] = true` for clarity.
6. In `VeHemi.increaseAmount()`, it would be more gas-efficient to inline the logic of `_increaseAmountFor()` (considering it is only used in this context) instead of calling it.
7. Instead of storing into `_lastPoint.blockNumber` up to two times between `VeHemi.sol#L521–528`, it can be more gas-efficient to do so only once by re-defining and re-implementing the branches.

8. In the NatSpec for `VeHemi.totalSupplyAtBlock()`, there is an outdated comment that states that the function is not yet implemented. However, the function logic is present. Consider removing the comment.

Recommendation: Consider applying the recommended improvements.

S2 Ownership Can Be Renounced

Acknowledged

Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

This is intended behaviour

File(s) affected: `VeHemi.sol`

Description: If the owner renounces their ownership, all ownable contracts will be left without an owner. Consequently, any function guarded by the `onlyOwner` modifier will no longer be able to be executed.

Recommendation: Confirm that this is the intended behavior. If not, override and disable the `renounceOwnership()` function in the `VeHemi` contract.

S3 Critical Role Transfer Not Following Two-Step Pattern

Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `4c662af30e0b12cc9ae49546a56d02343c235a60`.

File(s) affected: `VeHemi.sol`

Description: The owner of the contracts can call `transferOwnership()` to transfer the ownership to a new address. If an uncontrollable address is accidentally provided as the new owner address then the contract will no longer have an active owner, and functions with the `onlyOwner` modifier can no longer be executed.

Recommendation: Consider using OpenZeppelin's `Ownable2Step` contract to adopt a two-step ownership pattern in which the new owner must accept their position before the transfer is complete.

S4 Unlocked Pragma

Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `4c662af30e0b12cc9ae49546a56d02343c235a60`.

File(s) affected: `VeHemi.sol`, `VeHemiVoteDelegation.sol`

Related Issue(s): [SWC-103](#)

Description: Every Solidity file specifies in the header a version number of the format `pragma solidity (^)0.8.*`. The caret (`^`) before the version number implies an unlocked pragma, meaning that the compiler will use the specified version *and above*, hence the term "unlocked".

Recommendation: For consistency and to prevent unexpected behavior in the future, we recommend to remove the caret to lock the file onto a specific Solidity version.

S5 Specify Versions of Dependencies

Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `4c662af30e0b12cc9ae49546a56d02343c235a60`.

File(s) affected: `foundry.toml`

Description: Currently, although the project makes use of OpenZeppelin's repositories, it does not specify which versions of these repositories are to be used. However, this may lead to discrepancies in contract bytecode and functionality, as OpenZeppelin's contracts change over time across different releases.

Recommendation: Specify the versions of OpenZeppelin contract repositories to be used. As of writing these notes, the latest release is v5.4.0, so we recommend using this version.

Definitions

- **High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- **Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- **Informational** – The issue does not pose an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Appendix

File Signatures

The following are the SHA-256 hashes of the reviewed files. A file with a different SHA-256 hash has been modified, intentionally or otherwise, after the security review. You are cautioned that a different SHA-256 hash could be (but is not necessarily) an indication of a changed condition or potential vulnerability that was not within the scope of the review.

Files

- Repo: `https://github.com/hemilabs/veHEMI`
- `873...a3f ./eslinttrc.json`
 - `930...fe7 ./github/workflows/test.yml`
 - `f9c...d0d ./gitignore`
 - `42a...877 ./gitmodules`
 - `75b...535 ./nvmrc`
 - `be9...051 ./prettierignore`
 - `41c...139 ./prettierrc`
 - `6df...a52 ./solhint.json`
 - `231...189 ./vscode/settings.json`
 - `324...616 ./README.md`
 - `0f9...ecb ./foundry.toml`
 - `09d...734 ./package-lock.json`
 - `cc0...998 ./package.json`
 - `e8f...74e ./remappings.txt`
 - `7f1...bb1 ./src/VeHemi.sol`
 - `574...281 ./src/VeHemiVoteDelegation.sol`
 - `b32...1e2 ./src/interfaces/IRewardDistributor.sol`
 - `e6f...01d ./src/interfaces/IVeHemi.sol`
 - `de7...548 ./src/interfaces/IVeHemiVoteDelegation.sol`
 - `a0f...491 ./src/libraries/SafeCast.sol`
 - `0ea...94f ./src/storage/VeHemiDelegationStorageV1.sol`
 - `a51...8b8 ./src/storage/VeHemiStorageV1.sol`
 - `097...8d6 ./test/HemiVoteDelegation.t.sol`

- 079...135 ./test/VeHemi.t.sol
- f9d...88a ./test/mocks/MockERC20.sol
- 945...627 ./test/mocks/MockHemiVoteDelegation.sol

Test Suite Results

The codebase includes a test suite with 72 passing tests, ranging from unit tests of core functionality to integration tests covering multi-step scenarios. The suite also features fuzz testing of balance decay and other relevant behaviours. To run them, execute the following commands from the repository’s root directory:

```
forge install
forge test
```

Fix-Review Update: Six new tests were added to the test suite to cover new functionality and fixes for identified issues.

```
[·] Compiling...
[·] Compiling 64 files with Solc 0.8.29
[·] Solc 0.8.29 finished in 3.90s
Compiler run successful!

Ran 34 tests for test/HemiVoteDelegation.t.sol:TestVeHemiVoteDelegation
[PASS] testBasicDelegation() (gas: 1227172)
[PASS] testCantDelegateExpiredLock() (gas: 647316)
[PASS] testDelegateBySig() (gas: 1142310)
[PASS] testDelegateBySigExpiredSignature() (gas: 1120774)
[PASS] testDelegateBySigInvalidNonce() (gas: 1120318)
[PASS] testDelegateBySigInvalidSignature() (gas: 1098035)
[PASS] testDelegatedVotesAfterForfeit() (gas: 2916847)
[PASS] testDelegationCheckpoints() (gas: 1112440)
[PASS] testDelegationExpiration() (gas: 1623869)
[PASS] testDelegationNotOwned() (gas: 641905)
[PASS] testDelegationSwitching() (gas: 1856553)
[PASS] testDelegationToNonExistentToken() (gas: 19952)
[PASS] testDelegationToSameDelegatee() (gas: 1263811)
[PASS] testDelegationToSelf() (gas: 1443110)
[PASS] testDifferentExpirations() (gas: 1787258)
[PASS] testEvents() (gas: 740392)
[PASS] testExpirations() (gas: 738566)
[PASS] testExtendLockAfterExpiredDelegation() (gas: 9343745)
[PASS] testGetPastVotes() (gas: 1230636)
[PASS] testGetPastVotesFutureTimestamp() (gas: 639571)
[PASS] testMultipleDelegationsToSameDelegatee() (gas: 1735332)
[PASS] testMultipleDelegatorsWithExpirations() (gas: 2774381)
[PASS] testNoDelegationExpired() (gas: 14408)
[PASS] testReDelegation() (gas: 1288171)
[PASS] testReDelegationWhenAmountIncreased() (gas: 1116154)
[PASS] testReDelegationWhenAmountIncreasedByDelegator() (gas: 1347934)
[PASS] testReDelegationWhenLockExtendedByDelegator() (gas: 1385326)
[PASS] testRemoveDelegation() (gas: 1285705)
[PASS] testVoteAfterDelegateeLockExpire() (gas: 1683632)
[PASS] testVoteAfterForfeit() (gas: 1527473)
[PASS] testVotingPowerBeforeAndAfterDelegation() (gas: 1229819)
[PASS] testWhenNoExpirations() (gas: 1099771)
[PASS] testWriteNewCheckpointNoExpirations() (gas: 1098835)
[PASS] testWriteNewCheckpointWithExpirations() (gas: 1199360)
Suite result: ok. 34 passed; 0 failed; 0 skipped; finished in 613.55ms (795.90ms CPU time)

Ran 44 tests for test/VeHemi.t.sol:VeHemiTest
[PASS] testBalanceOfNFT() (gas: 519051)
[PASS] testCheckpointUpdatesUserPointHistory() (gas: 3547462)
[PASS] testCreateLock() (gas: 499598)
[PASS] testCreateLockFor() (gas: 482646)
[PASS] testDepositForIncreasesLockAmount() (gas: 574141)
[PASS] testERC721EnumerableFunctions() (gas: 820383)
[PASS] testEpoch() (gas: 695214)
[PASS] testExtendLockShouldNotExtendTransferable() (gas: 6759966)
```

```
[PASS] testForfeitAdminCanBeSetToZero() (gas: 33516)
[PASS] testForfeitAdminCanBeUpdated() (gas: 49193)
[PASS] testForfeitAdminUpdateRevertsIfNotOwner() (gas: 18163)
[PASS] testForfeitLockByAdmin() (gas: 491790)
[PASS] testForfeitLockEmitsCorrectEvents() (gas: 482124)
[PASS] testForfeitLockMixedWithRegularLocks() (gas: 760871)
[PASS] testForfeitLockRevertsIfLockExpired() (gas: 553632)
[PASS] testForfeitLockRevertsIfNotAdmin() (gas: 552812)
[PASS] testForfeitLockRevertsIfNotForfeitable() (gas: 534750)
[PASS] testForfeitLockTransfersTokensToAdmin() (gas: 487853)
[PASS] testForfeitLockUpdatesTotalVeHemiSupply() (gas: 798106)
[PASS] testForfeitLockWithZeroForfeitAdmin() (gas: 532897)
[PASS] testFuzz_BalanceOfNFT_TimeDecay(uint256,uint256,uint256) (runs: 1000, μ: 510025, ~: 510002)
[PASS] testFuzz_ForfeitLockWithDifferentAmounts(uint256) (runs: 1000, μ: 489661, ~: 489696)
[PASS] testFuzz_ForfeitLockWithDifferentDurations(uint256) (runs: 1000, μ: 487706, ~: 487739)
[PASS] testFuzz_TotalVeHemiSupplyAt_TimeConsistency(uint256,uint256,uint256,uint256,uint256) (runs: 1000, μ: 999353, ~: 875138)
[PASS] testFuzz_TotalVeHemiSupply_Consistency(uint256,uint256) (runs: 1000, μ: 833996, ~: 834065)
[PASS] testIncreaseUnlockTime() (gas: 2155693)
[PASS] testIncreaseUnlockTimeRevertsIfNotGreater() (gas: 495129)
[PASS] testIncreaseUnlockTimeRevertsIfNotOwner() (gas: 494503)
[PASS] testMultipleTransfersUpdateUserPointHistory() (gas: 814735)
[PASS] testNonTransferableNFT() (gas: 507657)
[PASS] testPastSupplyAt() (gas: 3388753)
[PASS] testSameBlock() (gas: 646098)
[PASS] testSupplyAt() (gas: 1361161)
[PASS] testTotalVeHemiSupply() (gas: 1061606)
[PASS] testTransferAfterExpiry() (gas: 6667177)
[PASS] testTransferAfterLockModificationUpdatesHistory() (gas: 690239)
[PASS] testTransferAllowedByDefault() (gas: 495658)
[PASS] testTransferNotAllowedFlag() (gas: 504791)
[PASS] testTransferUpdatesUserPointHistory() (gas: 665086)
[PASS] testUpdateRewardDistributor() (gas: 51890)
[PASS] testUpdateRewardDistributorToZero() (gas: 21129)
[PASS] testWithdraw() (gas: 530178)
[PASS] testWithdrawAfterExpiry() (gas: 532815)
[PASS] test_BalanceOfNFTAt() (gas: 512624)
Suite result: ok. 44 passed; 0 failed; 0 skipped; finished in 1.19s (2.82s CPU time)

Ran 2 test suites in 1.23s (1.81s CPU time): 78 tests passed, 0 failed, 0 skipped (78 total tests)
```

Code Coverage

The test suite achieves strong overall code coverage, with most categories scoring greater than 90%. However, branch coverage is notably lower, ranging from 65% to 77%. We recommend increasing branch coverage to at least 90% to further improve code quality.

Fix-Review Update: Although additional tests were added, the overall coverage score decreased. We recommend increasing test coverage to ensure all new functionality is properly tested.

File	% Lines	% Statements	% Branches	% Funcs
src/VeHemi.sol	91.67% (253/276)	87.63% (262/299)	63.24% (43/68)	93.94% (31/33)
src/VeHemiVoteDelegation.sol	92.64% (151/163)	93.05% (174/187)	76.00% (19/25)	83.33% (20/24)
test/mocks/MockERC20.sol	66.67% (4/6)	66.67% (2/3)	100.00% (0/0)	66.67% (2/3)
test/mocks/MockHemiVoteDelegation.sol	50.00% (4/8)	50.00% (2/4)	100.00% (0/0)	50.00% (2/4)
test/mocks/MockVeHemi.sol	100.00% (1/1)	100.00% (0/0)	100.00% (0/0)	100.00% (1/1)

File	% Lines	% Statements	% Branches	% Funcs
Total	90.97% (413/454)	89.25% (440/493)	66.67% (62/93)	86.15% (56/65)

Changelog

- 2025-07-25 - Initial report
- 2025-08-07 - Final report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp’s mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp’s team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 500 audits and secured over \$200 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Polygon, Avalanche, Curve, Solana, Compound, Lido, MakerDAO, Arbitrum, OpenSea and the World Economic Forum.

Quantstamp’s collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Near, Flow, Avalanche, Solana, Cardano, Binance Smart Chain, Hedera Hashgraph, Tezos
- DeFi: Curve, Compound, Maker, Lido, Polygon, Arbitrum, SushiSwap
- NFT: OpenSea, Parallel, Dapper Labs, Decentraland, Sandbox, Axie Infinity, Illuvium, NBA Top Shot, Zora
- Academic institutions: National University of Singapore, MIT

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee

its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

